

Padrões de Projeto (Observer)

QXD0068 - Reuso de Software

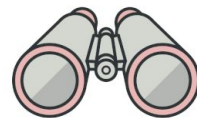
Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

- Introdução
- Padrão Observer
- Problema
- Solução
- Analogia do Mundo Real
- Estrutura do Padrão
- Exemplo em Kotlin e Python
- Aplicabilidade
- Prós e Contras
- Implementação



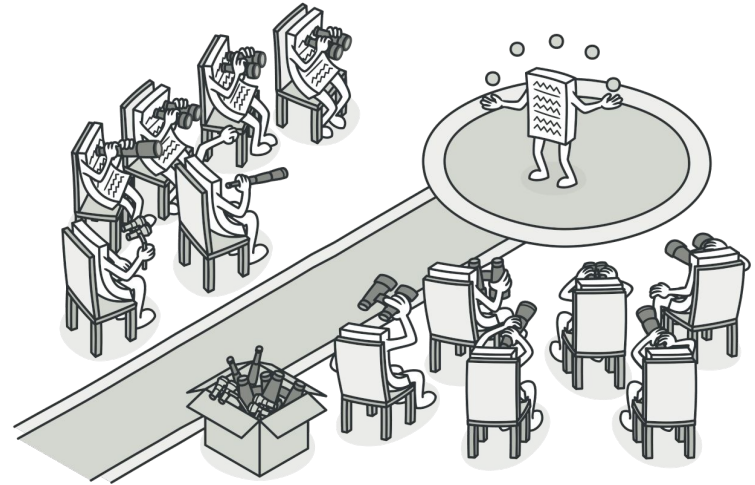
Observer

Introdução

- O Observer é um padrão de projeto comportamental que define um mecanismo de assinatura/notificação entre objetos.
- Permite que múltiplos objetos (assinantes) sejam automaticamente notificados sobre mudanças no estado de outro objeto (publicador/sujeito).

Padrão Observer

- O padrão Observer serve para notificar vários objetos sobre mudanças em um objeto central, estabelecendo uma relação "um-para-muitos" onde o objeto "sujeito" (ou "publicador") notifica "observadores" (ou "assinantes") automaticamente quando seu estado é alterado.

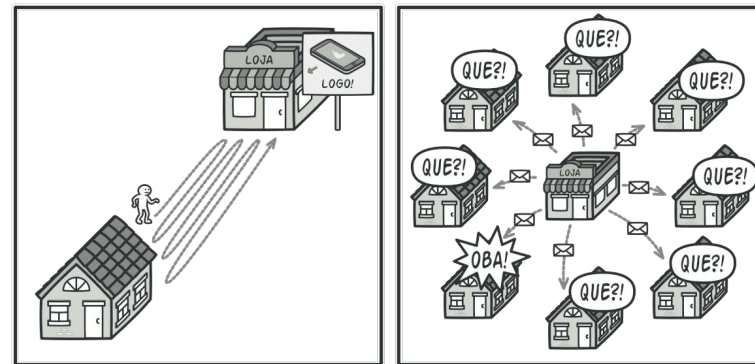


Problema

- Imagine que você tem dois tipos de objetos: um Cliente e uma Loja. O cliente está muito interessado em uma marca particular de um produto (digamos que seja um novo modelo de iPhone) que logo deverá estar disponível na loja.
- O cliente pode visitar a loja todos os dias e checar a disponibilidade do produto. Mas enquanto o produto ainda está a caminho, a maioria desses visitas serão em vão.
- Por outro lado, a loja poderia mandar milhares de emails (que poderiam ser considerados como spam) para todos os clientes cada vez que um novo produto se torna disponível. Isso salvaria alguns clientes de incontáveis viagens até a loja. Porém, ao mesmo tempo, irritaria outros clientes que não estão interessados em novos produtos.
- Parece que temos um conflito. Ou o cliente gasta tempo verificando a disponibilidade do produto ou a loja gasta recursos notificando os clientes errados.

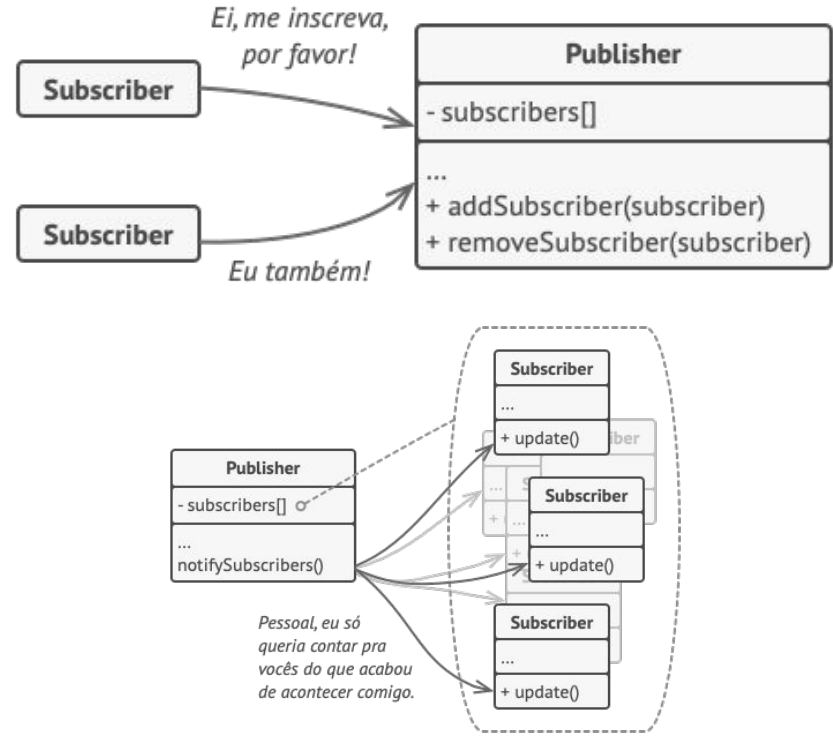
Problema

- **Exemplo:** um cliente quer saber quando um produto chega na loja.
 - Se ele visitar a loja todos os dias → perda de tempo.
 - Se a loja avisar todos os clientes → spam.
- Solução ideal: somente os interessados recebem notificações, no momento certo.



Solução

- Introduz um mecanismo de assinatura na classe publicadora.
- Assinantes podem inscrever-se ou cancelar a inscrição.
- Quando algo muda, o publicador notifica todos os assinantes através de um método comum (ex.: `update()`).



Analogia do Mundo Real

- **Como uma assinatura de jornal ou revista:**
 - Você não precisa verificar toda hora se há nova edição.
 - A editora (publicadora) envia automaticamente as novas edições aos assinantes.
 - Os assinantes podem cancelar quando quiserem.

Analogia do Mundo Real



Estrutura do Padrão

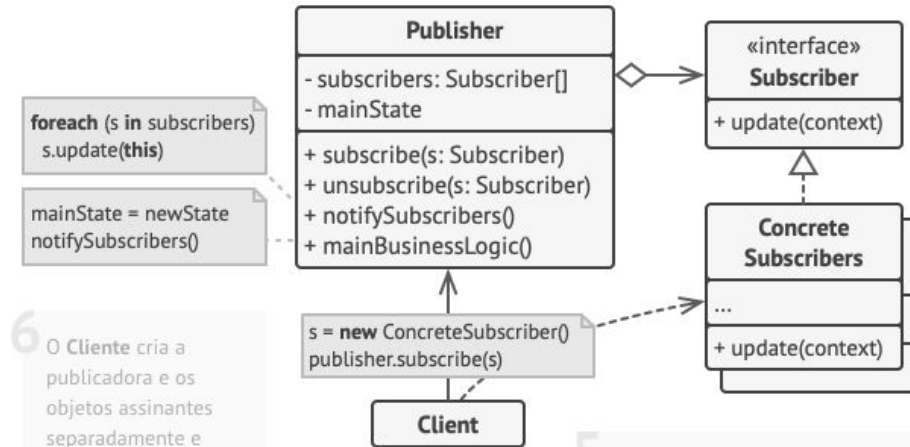
- **Publicadora (Subject):** mantém lista de assinantes e envia notificações.
- **Assinante (Observer):** define a interface update().
- **Assinantes Concretos:** implementam reações específicas à notificação.
- **Cliente:** cria publicadoras e assinantes e registra as assinaturas.
- **Fluxo:**
 - Publicadora muda estado → chama update() em cada Assinante registrado.

1 A **Publicadora** manda eventos de interesse para outros objetos. Esses eventos ocorrem quando a publicadora muda seu estado ou executa algum comportamento. As publicadoras contêm uma infraestrutura de inscrição que permite novos assinantes se juntar aos atuais assinantes ou deixar a lista.

2 Quando um novo evento acontece, a publicadora percorre a lista de assinantes e chama o método de notificação declarado na interface do assinante em cada objeto assinante.

3 A interface do **Assinante** declara a interface de notificação. Na maioria dos casos ela consiste de um único método `atualizar`. O método pode ter vários parâmetros que permite que a publicadora passe alguns detalhes do evento junto com a atualização.

4 **Assinantes Concretos** realizam algumas ações em resposta às notificações enviadas pela publicadora. Todas essas classes devem implementar a mesma interface para que a publicadora não fique acoplada às classes concretas.



6 O **Cliente** cria a publicadora e os objetos assinantes separadamente e então registra os assinantes para as atualizações da publicadora.

5 Geralmente, assinantes precisam de alguma informação contextual para lidar com a atualização corretamente. Por esse motivo, as publicadoras quase sempre passam algum dado de contexto como argumentos do método de notificação. A publicadora pode passar a si mesmo como um argumento, permitindo que o assinante recupere quaisquer dados diretamente.

```
// Interface Observer (assinante)
interface Observer {
    fun update(message: String)
}

// Interface Subject (publicador)
interface Subject {
    fun addObserver(observer: Observer)
    fun removeObserver(observer: Observer)
    fun notifyObservers(message: String)
}
```

```
// Implementação concreta do Subject
class Loja : Subject {
    private val observers = mutableListOf<Observer>()

    override fun addObserver(observer: Observer) {
        observers.add(observer)
    }

    override fun removeObserver(observer: Observer) {
        observers.remove(observer)
    }

    override fun notifyObservers(message: String) {
        for (observer in observers) {
            observer.update(message)
        }
    }

    fun novoProduto(nome: String) {
        println("Loja: Novo produto chegou -> $nome")
        notifyObservers("Produto disponível: $nome")
    }
}
```

Exemplo em Kotlin

```
// Observadores concretos
class ClienteEmail(private val nome: String) : Observer {
    override fun update(message: String) {
        println("[${nome} - Email] Recebeu notificação: $message")
    }
}

class ClienteSMS(private val nome: String) : Observer {
    override fun update(message: String) {
        println("[${nome} - SMS] Recebeu mensagem: $message")
    }
}

// Uso prático
fun main() {
    val loja = Loja()
    val cliente1 = ClienteEmail("Ana")
    val cliente2 = ClienteSMS("Bruno")

    loja.addObserver(cliente1)
    loja.addObserver(cliente2)

    loja.novoProduto("iPhone 20 Ultra")
    loja.novoProduto("MacBook Air M5")
}
```

Exemplo em Kotlin

```
# Observer (assinante)
class Observer:
    def update(self, message):
        pass

# Subject (publicador)
class Subject:
    def __init__(self):
        self._observers = []

    def attach(self, observer: Observer):
        self._observers.append(observer)

    def detach(self, observer: Observer):
        self._observers.remove(observer)

    def notify(self, message):
        for observer in self._observers:
            observer.update(message)

# Observadores concretos
class EmailObserver(Observer):
    def update(self, message):
        print(f"[Email] Notificação recebida: {message}")

class SMSObserver(Observer):
    def update(self, message):
        print(f"[SMS] Mensagem recebida: {message}")
```

Exemplo em Python

```
# Uso prático
if __name__ == "__main__":
    loja = Subject()

    cliente1 = EmailObserver()
    cliente2 = SMSObserver()

    loja.attach(cliente1)
    loja.attach(cliente2)

    loja.notify("Novo produto disponível: iPhone 20 Ultra!")
    loja.notify("Promoção relâmpago! 20% de desconto por 24h.")
```

Aplicabilidade

- **Use o padrão Observer quando:**
 - Mudanças em um objeto exigem atualização automática de outros.
 - O conjunto de dependentes muda dinamicamente.
 - É necessário desacoplar componentes (ex.: GUI, eventos de botões, etc.).

Prós e Contras

- **Vantagens**

- Baixo acoplamento entre objetos.
- Extensível: novos assinantes sem alterar publicadora.
- Flexível: relação dinâmica em tempo de execução.

- **Desvantagens**

- Ordem de notificação não garantida.
- Pode ser difícil de depurar se houver muitas dependências dinâmicas.

Projeto Real

- Trivia Refactoring Kata + Observer
 - <https://github.com/victorrentea/kata-trivia-java>

 master ▾
  8 Branches
  0 Tags

 victorrentea Release it into the public domain	62e241e · last month	 51 Commits
 art	art	4 years ago
 src	polish	last year
 .gitignore	ignore	4 years ago
 LICENSE	Release it into the public domain	last month
 pom.xml	Revert "fun"	2 years ago
 readme.md	rename	2 years ago

Contributors 4

-  **victorrentea** Victor Rentea
-  **alexbatt1980** Alex Batt
-  **mariopiogioiosa** Mario Pio Gioiosa
-  **mibalan** Mihai Balan

Languages

● Java 100%

Projeto Real

- A classe Game.java mistura a lógica do jogo com várias chamadas diretas
- E se além do console quisermos enviar os acontecimentos do jogo para uma interface gráfica, arquivo de log ou sistema de estatísticas?

```

1  package trivia;
2
3  import java.util.ArrayList;
4  import java.util.LinkedList;
5
6  // REFACTOR ME
7  public class Game implements IGame {
8      ArrayList players = new ArrayList();
9      int[] places = new int[6];
10     int[] purses = new int[6];
11     boolean[] inPenaltyBox = new boolean[6];
12
13     LinkedList popQuestions = new LinkedList();
14     LinkedList scienceQuestions = new LinkedList();
15     LinkedList sportsQuestions = new LinkedList();
16     LinkedList rockQuestions = new LinkedList();
17
18     int currentPlayer = 0;
19     boolean isGettingOutOfPenaltyBox;
20
21     public Game() {
22         for (int i = 0; i < 50; i++) {
23             popQuestions.addLast("Pop Question " + i);
24             scienceQuestions.addLast("Science Question " + i);
25             sportsQuestions.addLast("Sports Question " + i);
26             rockQuestions.addLast(createRockQuestion(i));
27         }
28     }
29
30     public String createRockQuestion(int index) {
31         return "Rock Question " + index;
32     }
33
34     public boolean isPlayable() {
35         return (howManyPlayers() >= 2);
36     }

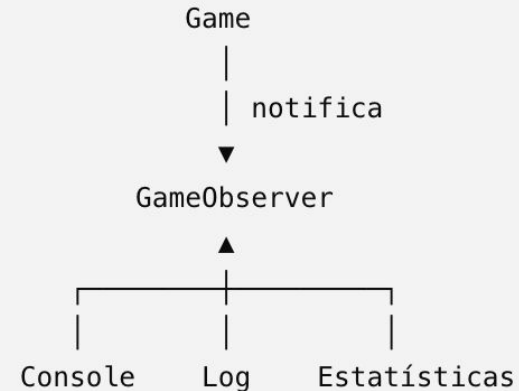
```

Projeto Real

- Com Observer, podemos chegar a algo assim:
- Depois

Game

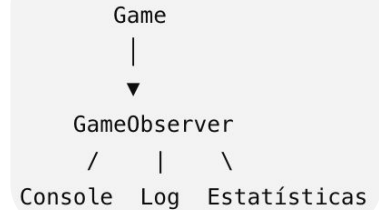
- |— lógica do jogo
- |— System.out.println()
- |— System.out.println()
- |— System.out.println()



Refatoração

```
src/main/java/trivia/
```

```
|  
├─ Game.java           ← código legado  
├─ GameOld.java        ← não alterar  
├─ IGame.java  
├─ PlayGame.java  
├─  
└─ observer/  
    ├─ GameObserver.java  
    ├─ ConsoleGameObserver.java  
    └─ LogGameObserver.java
```



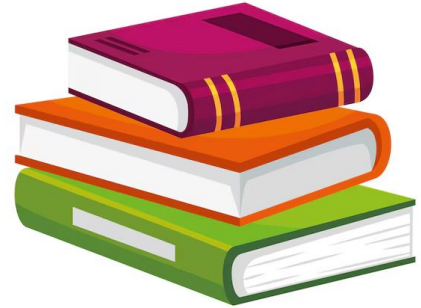
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado!

Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

